

LAPORAN UAS
TUGAS PROYEK AKHIR TEKNIK KOMPILASI INI AKAN
MENGIMPLEMENTASIKAN KONSTRUKSI PENANGANAN
ERROR (ERROR HANDLER: TRY-CATCH) MENGGUNAKAN
BAHASA PEMROGRAMAN PYTHON



TEKNIK KOMPILASI

Disusun Oleh:

SOMANTRI - 221011401711

FAKULTAS ILMU KOMPUTER
PROGRAM STUDI TEKNIK INFORMATIKA S1
UNIVERSITAS PAMULANG
2025/2026

1. Kode Implementasi Lengkap (Bagian 1, 2, & 3)

Berikut adalah kode utuh MiniCompiler setelah menyelesaikan ketiga bagian TUGAS di dalam fungsi tokenisasi, power(), dan term():

```
import re

# =====

# 1. DEFINISI NODE AST

# =====

class AST:

    pass

class BinOp(AST):

    def __init__(self, left, op, right):

        self.left = left

        self.op = op

        self.right = right

class Num(AST):

    def __init__(self, value):

        self.value = value

class Var(AST):

    def __init__(self, name):

        self.name = name

class ParserError(Exception):

    pass

# =====

# 2. IMPLEMENTASI MINICOMPILER

# =====

class MiniCompiler:
```

```

def __init__(self, source, env):
    # TUGAS 1: Menambahkan simbol '^' ke dalam regex di akhir kumpulan karakter [+*/()\^~]
    -> [+*/()\^~]

    self._tokens = iter(re.findall(r'[a-zA-Z_]\w*|d+(?:\.\d+)?|[+*/()\^~]', source) + ['?'])
    self._current = None
    self._env = env
    self._temp_count = 0
    self.advance()

def advance(self):
    try:
        self._current = next(self._tokens)
    except StopIteration:
        self._current = None

def expect(self, expected):
    if self._current != expected and not (expected == "ID" and self._current.isalnum()):
        raise ParserError(f'Expected {expected}, found {self._current}')

    token = self._current
    self.advance()
    return token

def factor(self):
    token = self._current

    if token is not None and token.replace('.', '', 1).isdigit():
        self.advance()
        return Num(float(token) if '.' in token else int(token))

    elif token and token.isalpha():
        if token not in self._env:
            raise ParserError(f'Semantic Error: Undefined variable '{token}')

        self.advance()
        return Var(token)

```

```

elif token == '(':
    self.advance()
    node = self.expr()
    self.expect(')')
    return node
raise ParserError(f"Unexpected token: {token}")

```

TUGAS 2: Implementasi fungsi power() untuk menangani operator '^'

```

def power(self):
    node = self.factor()
    while self._current == '^':
        op = self._current
        self.advance()
        # Operator pangkat umumnya bersifat kanan-ke-kiri (right-associative),
        # namun sesuai pola linear term(), pengerjaan di bawah menggunakan left-associative.
        node = BinOp(left=node, op=op, right=self.factor())
    return node

```

def term(self):

```

# TUGAS 3: Hubungkan hierarki tertinggi ke self.power() bukan ke self.factor()
node = self.power()
while self._current in ('*', '/'):
    op = self._current
    self.advance()
    # Sesuaikan pemanggilan sisi kanannya ke self.power()
    node = BinOp(left=node, op=op, right=self.power())
return node

```

def expr(self):

```

node = self.term()
while self._current in ('+', '-'):
    op = self._current

```

```

        self.advance()

        node = BinOp(left=node, op=op, right=self.term())

    return node

def generate_tac(self, node):
    if isinstance(node, Num): return str(node.value)
    if isinstance(node, Var): return node.name

    left_val = self.generate_tac(node.left)
    right_val = self.generate_tac(node.right)

    self._temp_count += 1
    temp_name = f't{self._temp_count}'
    print(f'{temp_name} = {left_val} {node.op} {right_val}')
    return temp_name

# =====
# 3. UJI COBA JALANNYA PROGRAM
# =====

source_code = "a ^ 2 + b * c"
symbol_table = {'a': 5, 'b': 10, 'c': 2}

try:
    print(f"Input: {source_code}")
    compiler = MiniCompiler(source_code, symbol_table)
    ast_root = compiler.expr()

    print("\n--- Output Three Address Code (TAC) ---")
    compiler.generate_tac(ast_root)
except Exception as e:
    print(f"Error: {e}")

Hasil Eksekusi Output TAC:

```

Jika Anda menjalankan kode di atas, program akan sukses menghasilkan output urutan instruksi linear tingkat rendah berikut:

$$t1 = a ^ 2$$
$$t2 = b * c$$
$$t3 = t1 + t2$$

2. Jawaban Pertanyaan Refleksi (Bagian 4)

Pertanyaan 1:

Mengapa fungsi `power()` harus dipanggil di dalam `term()`, bukan sebaliknya? Jelaskan kaitannya dengan *Operator Precedence*.

Jawaban: Dalam teknik kompilasi dengan pendekatan *Recursive Descent Parsing*, hierarki prioritas operator (*operator precedence*) berbanding terbalik dengan kedalaman struktur pemanggilan fungsi fungsi tata bahasa (*grammar production*). Semakin tinggi prioritas suatu operator matematika, ia harus diletakkan di lapisan fungsi yang paling dalam (mendekati elemen terkecil/factor).

Karena operator pangkat (^) memiliki tingkat prioritas yang lebih tinggi daripada perkalian/pembagian (*, /), maka `power()` diposisikan di dalam mengevaluasi factor terlebih dahulu sebelum `term()` mengevaluasi operator di atasnya. Jika susunannya dibalik (`term()` dipanggil di dalam `power()`), maka operasi perkalian akan mengikat operand lebih kuat daripada pangkat. Hal itu keliru dan merusak aturan baku operasi aritmetika matematika komputer.

Pertanyaan 2:

Apa yang terjadi pada fase Analisis Semantik jika variabel `z` digunakan dalam kode sumber tetapi tidak ada di `symbol_table`?

Jawaban: Ketika kode sumber mendefinisikan variabel `z`, fungsi `factor()` pada kompilator akan memeriksa keberadaan nama variabel tersebut di dalam lingkungan tabel simbol (`self._env`). Karena `z` tidak terdaftar atau tidak dideklarasikan di dalam `symbol_table`, kompilator akan mendeteksi pelanggaran aturan konteks bahasa pemrograman dan langsung melempar interupsi galat berupa `ParserError("Semantic Error: Undefined variable 'z'")`.

Secara struktural, kode tersebut mungkin lolos dalam fase analisis sintaksis (karena pola strukturnya benar sebagai penanda/identifier), namun kompilasi gagal total di fase analisis semantik karena variabel tersebut tidak memiliki kepastian makna alokasi nilai atau rujukan data di memori runtime sistem.

Pertanyaan 3:

Jelaskan mengapa dalam TAC, instruksi untuk $a ^ 2$ harus muncul sebelum instruksi untuk $+$.

Jawaban: Representasi *Three-Address Code* (AST ke TAC) bekerja secara linier dengan cara melakukan penelusuran pohon sintaksis objek secara mendalam pasca-order (*Post-order Tree*

Traversal) dari bawah ke atas. Berdasarkan kaidah struktur pohon sintaksis abstrak (AST) yang dibangun, node operasi pangkat $^$ (dengan anak a dan 2) berada di tingkat percabangan terdalam, sedangkan operasi penjumlahan $+$ berada di posisi node akar (*root node*) paling atas karena memiliki prioritas terendah dalam ekspresi tersebut.

Kompilator tidak akan pernah bisa mengeksekusi operasi penambahan $+$ sebelum nilai kiri dan nilai kanannya mutlak diketahui secara atomik. Oleh sebab itu, sub-pohon dengan prioritas tertinggi (a^2) wajib dievaluasi terlebih dahulu menjadi variabel temporer $t1$. Setelah nilai komputasi antara $t1$ dan $t2$ ($b * c$) selesai diproduksi, barulah instruksi penjumlahan $+$ dapat dijalankan dengan menggunakan nilai operan penampung tersebut ($t3 = t1 + t2$).